Complemento Práctico

Conceptos extendidos basados en las clases audiovisuales

Este documento recopila explicaciones detalladas, consejos de depuración y configuraciones avanzadas del entorno que se discutieron en las clases grabadas. A diferencia de los apuntes teóricos, aquí nos enfocaremos en la **práctica real**: por qué ocurren ciertos errores de compilación "invisibles", cómo aprovechar las sugerencias modernas de NetBeans y cómo configurar el entorno para ver lo que otros no ven.

Contenido

El Ciclo de Vida de Java

La estrategia detrás de las versiones LTS.

Sintaxis Fina

Errores de espacios y asignaciones que rompen la lógica.

Modern Java

La evolución del switch y las sugerencias del IDE.

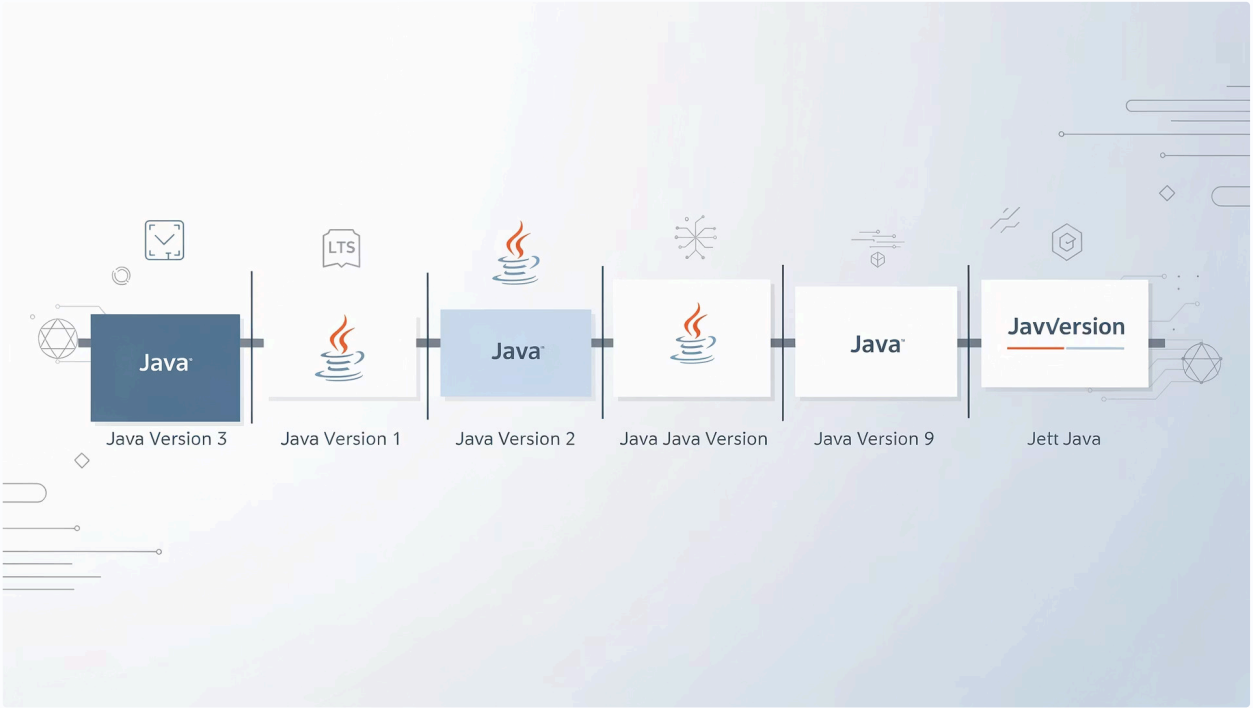
Depuración (Debugging)

Cómo ver la memoria en tiempo real.

Documentación Oculta

Configuración avanzada de Javadoc.

Gestión de Versiones Java (JDK)



En los apuntes leemos "instalar Java", pero en la clase en vivo profundizamos en el criterio de selección profesional. Oracle mantiene un ritmo de actualización agresivo, lanzando dos versiones al año (marzo y septiembre). Esto puede confundirnos: ¿Instalo la 19, la 20, la 22?

La Estrategia LTS (Long Term Support)

No debemos perseguir siempre el número más alto. Las versiones intermedias (como la 18, 19, 20) son versiones de transición con una vida útil de apenas 6 meses. Nosotros, al igual que la industria bancaria y corporativa, priorizamos las versiones **LTS** (como la **17** o la **21**).

¿Por qué?

Porque garantizan soporte de seguridad y estabilidad por 5 a 8 años. Nadie quiere reescribir su software cada 6 meses porque la versión de Java caducó.

Consejo de Instalación

Al entrar a la web de Oracle, ignoren las versiones "novedosas" y busquen siempre la etiqueta "LTS" al lado del número. Esa es la opción segura para cursar y trabajar.

Sutilezas de los Operadores: El "Error del Espacio"

Un error de compilación muy frustrante para los principiantes ocurre con los operadores relacionales compuestos (>=, <=, !=). El compilador de Java analiza el código buscando "tokens" (palabras o símbolos reconocibles).

El Problema

Si nosotros escribimos > = con un espacio en el medio, Java ve dos símbolos desconectados: un "mayor que" y un "asignación". No sabe cómo interpretarlos juntos y lanza un error de sintaxis que a veces es difícil de detectar a simple vista.

Ejemplo de Código (Incorrecto vs Correcto)

```
int a = 10;
int b = 5;

// INCORRECTO: El espacio genera error de compilación
// if (a > = b) { ... } <-- ERROR

// CORRECTO: Los caracteres deben estar pegados
if (a >= b) {
   System.out.println("Es mayor o igual");
}
```

Asignación vs. Comparación

Reiteramos este punto porque es la causa número uno de errores lógicos en los exámenes. En matemáticas, el signo = significa igualdad. En programación, significa **destrucción y asignación**. Si usamos un solo = dentro de un if, no estamos preguntando; estamos afirmando.

Análisis del Error

```
boolean encendido = false;

// ERROR GRAVE: Usamos un solo =
// Esto ASIGNA true a la variable y luego evalúa si es verdadero.
// Resultado: Siempre entra al if, sin importar el valor original.
if (encendido = true) {
    System.out.println("El sistema se encendió");
}

// FORMA CORRECTA: Usamos doble == para comparar
if (encendido == true) {
    ...
}
```

> *Tip:* Incluso mejor, para booleanos, usen simplemente `if (encendido)`.

La Evolución del Switch (Rule Switch)

Si están utilizando un JDK moderno (versión 12+), verán que NetBeans les muestra una "lamparita amarilla" al lado de sus estructuras switch tradicionales. El IDE nos está sugiriendo modernizar el código.

¿Qué es el "Rule Switch" o Switch de Flecha?

Es una sintaxis simplificada que soluciona el viejo problema del break olvidado. En el switch clásico, si olvidábamos el break, el programa seguía ejecutando los casos de abajo (fall-through). Con la flecha (->), esto es imposible.

Comparativa de Evolución

```
int opcion = 1;

// 1. SWITCH CLÁSICO (Verboso y propenso a error si falta break)
switch (opcion) {
    case 1:
        System.out.println("Uno");
        break; // ¡Obligatorio!
    case 2:
        System.out.println("Dos");
        break;
}

// 2. RULE SWITCH (Moderno, Java 14+)
// Más limpio, sin breaks, imposible equivocarse
switch (opcion) {
    case 1 -> System.out.println("Uno");
    case 2 -> System.out.println("Dos");
}
```

Nota: Aunque es tentador usarlo, recuerden que el switch clásico sigue siendo el más compatible con código antiguo (Legacy).

Debugging: Ver lo Invisible

Cuando explicamos ciclos (for, while), mencionamos el "Teorema de la Programación Estructurada". Pero, ¿qué pasa cuando el ciclo no termina nunca (bucle infinito) o suma mal? Leer el código estáticamente a veces no alcanza.

La Técnica del Breakpoint en NetBeans

En los videos mostramos cómo "congelar" el tiempo:

01

Establecer el Breakpoint

Hagan clic en el margen gris izquierdo (sobre el número de línea). Aparecerá un **cuadrado rojo**.

02

Iniciar el Debug

En lugar de dar "Play" (triángulo verde), den clic al botón de **Debug** (bug/bicho).

03

Observar la Ejecución

El programa se detendrá en esa línea.

¿Qué ganamos?

Podemos pasar el mouse sobre las variables (i, contador, acumulador) y ver su valor exacto en la memoria RAM en ese instante. Es la única forma real de entender el flujo paso a paso.

Configurando Javadoc en NetBeans

Los apuntes enseñan a escribir /** comentarios */. Pero hay un truco de NetBeans que no suele estar en los libros. Por defecto, cuando generamos la web de documentación (Run > Generate Javadoc), NetBeans **solo muestra las clases y métodos públicos**. Oculta todo lo privado.

¿Cómo documentar para estudiar?

Si ustedes quieren ver sus propios métodos privados en la web generada para estudiar su lógica interna:

Abrir Propiedades

Clic derecho en el proyecto -> **Properties**.

Categoría Documenting

Categoría **Documenting**.

Activar Opción

Activar el checkbox: **"Include Private and Package Private Members"**.

Al regenerar, verán explicados hasta los métodos internos más pequeños, lo cual es ideal para el aprendizaje académico, aunque en el mundo profesional se suele ocultar.

Resumen de Trucos de Experto

Versiones LTS

Priorizar estabilidad sobre novedad. Java 17 y 21 son las opciones profesionales.

Operadores sin Espacios

Los símbolos >= <= != deben escribirse pegados para evitar errores de compilación.

Doble Igual para Comparar

Usar == para comparar, = solo asigna valores. Error lógico número uno en exámenes.

Switch Moderno

La sintaxis con flecha (->) elimina el problema del break olvidado en Java 14+.

Breakpoints

Usar el debugger para ver valores de variables en tiempo real, no adivinar.

Javadoc Completo

Activar documentación de métodos privados en Properties > Documenting para estudiar.

Recuerden: Estos trucos provienen de la experiencia práctica en las clases. Dominarlos les dará ventaja tanto en los exámenes como en proyectos reales.